

Kiến Trúc Next.js App Router

Cẩm nang hướng dẫn toàn diện hệ thống File-system Routing thế hệ mới

Next.js App Router giới thiệu một mô hình định tuyến hoàn toàn mới dựa trên các tính năng tân tiến của React, nổi bật nhất là **React Server Components (RSC)**. Hệ thống định tuyến này tổ chức các route thông qua cấu trúc thư mục trong phân vùng `app/` và sử dụng các file đặc biệt nhằm kiến tạo giao diện một cách đồng bộ, tối ưu hiệu năng và hỗ trợ SEO vượt trội.

1. Các File Đặc Biệt Trong Định Tuyến (Routing Files)

Trong App Router, mỗi file đặc biệt nằm trong một phân đoạn đường dẫn (route segment) đảm nhận một vai trò duy nhất trong việc cấu trúc nên vòng đời UI của ứng dụng:

Trang Định (`page.tsx`) SERVER COMPONENT

Là giao diện cốt lõi hiển thị trực tiếp với người dùng khi truy cập URL. Mặc định là một Server Component, cho phép thực thi logic bất đồng bộ (`async/await`) để fetch dữ liệu trực tiếp tại phía máy chủ mà không cần sử dụng tới `useEffect` hoặc các hook quản lý trạng thái phức tạp ở Client.

```
app/dashboard/page.tsx

export default async function DashboardPage() {
  // Fetch dữ liệu trực tiếp tại Server hiệu quả và bảo mật
  const data = await fetch('https://api.example.com/stats');
  const stats = await data.json();

  return (
    <main>
      <h1>Dashboard Overview</h1>
      <p>Active Users: {stats.count}</p>
    </main>
  );
}

export const metadata = {
  title: 'Dashboard',
  description: 'Tổng quan phân tích số liệu hệ thống.',
};
```

Trạng Thái Tải Dữ Liệu (`loading.tsx`) AUTO SUSPENSE

File này tự động bọc phân đoạn trang tương ứng bên trong cơ chế `<Suspense>` của React. Trạng thái Loading sẽ được hiển thị ngay lập tức khi người dùng chuyển hướng trang, giúp tối ưu hóa cảm giác mượt mà (perceived performance) của hệ thống.

app/dashboard/loading.tsx

```
export default function DashboardLoading() {
  return (
    <div className="loading-skeleton">
      <div className="skeleton-title" />
      <div className="skeleton-body" />
    </div>
  );
}
```

Xử Lý Ngoại Lệ (error.tsx) CLIENT COMPONENT

Đóng vai trò là một **React Error Boundary** cô lập lỗi phát sinh. Khi có lỗi xảy ra trong segment đó, ứng dụng sẽ không bị sập toàn bộ mà chỉ hiển thị giao diện thay thế tại khu vực bị lỗi, đi kèm cơ chế khôi phục nhanh qua hàm `reset()`.

app/dashboard/error.tsx

```
'use client'; // Bắt buộc phải là Client Component

export default function DashboardError({
  error,
  reset,
}: {
  error: Error;
  reset: () => void;
}) {
  return (
    <div className="error-container">
      <h2>Đã xảy ra sự cố dữ liệu!</h2>
      <p>{error.message}</p>
      <button onClick={reset}>Thử tải lại trang</button>
    </div>
  );
}
```

Điểm Đầu Cuối API (route.ts) API HANDLER

Dùng để xây dựng các API endpoint nội bộ (RESTful API), xử lý các phương thức HTTP cơ bản như `GET`, `POST`, `PUT`, `DELETE`. Lưu ý: Không được đặt file `route.ts` chung thư mục với file `page.tsx` nhằm tránh xung đột tài nguyên.

app/api/users/route.ts

```
import { NextRequest, NextResponse } from 'next/server';

export async function GET(req: NextRequest) {
  const { searchParams } = req.nextUrl;
  const page = searchParams.get('page') ?? '1';
  const users = await db.users.findMany({ page });
  return NextResponse.json(users);
}

export async function POST(req: NextRequest) {
  const body = await req.json();
  const newUser = await db.users.create(body);
  return NextResponse.json(newUser, { status: 201 });
}
```

Không Tìm Thấy Trang (not-found.tsx)

Được render tự động khi không tìm thấy phân đoạn đường dẫn phù hợp hoặc khi hàm xử lý `notFound()` từ thư viện `next/navigation` được kích hoạt một cách chủ động trong logic nghiệp vụ.

app/dashboard/not-found.tsx

```
import { notFound } from 'next/navigation';

async function getUser(id: string) {
  const user = await db.users.findById(id);
  if (!user) notFound(); // Trigger hiển thị giao diện không tìm thấy
  return user;
}

export default function NotFound() {
  return (
    <div>
      <h2>404 - Thành viên không tồn tại</h2>
      <a href="/dashboard">Quay lại Bảng điều khiển</a>
    </div>
  );
}
```

2. Định Tuyến Lồng Nhau & Cơ Chế Layout Kế Thừa (Nested Routes)

Next.js sử dụng triết lý phân cấp thư mục để định hình URL. Khi các thư mục lồng nhau, cấu trúc UI (Layout) cũng sẽ lồng ghép tương ứng một cách tự động.

```

app/
├─ layout.tsx /* Root Layout */
├─ page.tsx → /
├─ blog/
│   ├─ layout.tsx /* Blog Layout */
│   ├─ page.tsx → /blog
│   └─ posts/
│       ├─ layout.tsx
│       └─ page.tsx → /blog/posts

```



ĐẶC TÍNH TỐI ƯU HÓA HIỆU NĂNG (PARTIAL RENDERING)

Khi điều hướng giữa các đường dẫn lồng nhau (ví dụ từ `/blog` sang `/blog/posts`), Next.js chỉ render lại duy nhất phần thay đổi thuộc file `page.tsx`. Các phân đoạn `Root Layout` và `Blog Layout` được bảo toàn trạng thái, hoàn toàn không bị re-render.

3. Phân Đoạn Đường Dẫn Động (Dynamic Routes)

Khi một đường dẫn cụ thể chưa được xác định trước, việc sử dụng Dynamic Segments giúp liên kết dữ liệu linh hoạt thông qua tham số (`params`).

Cấu trúc thư mục	Ví dụ URL	Giá trị biến tương ứng <code>params</code>
<code>app/blog/[slug]/page.tsx</code>	<code>/blog/huong-dan-nextjs</code>	<code>{ slug: "huong-dan-nextjs" }</code>
<code>app/shop/[...slug]/page.tsx</code>	<code>/shop/ao/nam/somi</code>	<code>{ slug: ["ao", "nam", "somi"] }</code> (<i>Catch-all</i>)
<code>app/docs/[...slug]/page.tsx</code>	<code>/docs</code> (<i>Không cần segment</i>)	<code>{ slug: undefined }</code> (<i>Optional Catch-all</i>)

Pre-render Tĩnh Với `generateStaticParams`

Để tối ưu hóa tốc độ tải trang, hàm này cho phép chuyển đổi các Dynamic Route thành dạng Static Site Generation (SSG). Next.js sẽ tiến hành biên dịch và tạo sẵn file HTML tĩnh ngay tại thời điểm build hệ thống.

●●● Cơ chế sinh trang tĩnh SSG

```
export async function generateStaticParams() {
  const posts = await getPosts();

  return posts.map((post) => ({
    slug: post.slug, // Pre-render trước toàn bộ danh sách bài viết dữ liệu
  }));
}

// Kiểm soát hành vi khi slug mới xuất hiện:
export const dynamicParams = false;
// false = Trả về 404 ngay lập tức nếu slug không tồn tại từ trước
// true  = Thực hiện cơ chế SSR on-demand để xử lý slug mới (Mặc định)
```

4. Nhóm Đường Dẫn (Route Groups) & Thư Mục Riêng Tư (Private Folders)

Hỗ trợ nhà phát triển tổ chức, phân chia cấu trúc dự án khoa học mà không làm thay đổi trực tiếp cấu hình hiển thị của đường dẫn URL.

Route Groups (Thư mục đóng ngoặc đơn: `(tên_thư_mục)`)

Giúp nhóm các route có chung tính chất, phục vụ tốt cho việc áp dụng các Layout chuyên biệt (ví dụ: tách biệt hoàn toàn giao diện phân hệ Đăng nhập và phân hệ Trang quản trị chính).

●●● Cấu trúc Module độc lập

```
app/
├── (auth)/                ← Không xuất hiện trên URL công khai
│   ├── layout.tsx        ← Giao diện tối giản dành cho Auth
│   ├── login/page.tsx    → /login
│   └── register/page.tsx
└── (dashboard)/
    ├── layout.tsx        ← Giao diện phức tạp (Sidebar + Nav)
    └── home/page.tsx     → /home
```

Private Folders (Thư mục bắt đầu bằng dấu gạch dưới: `_tên_thư_mục`)

Loại bỏ hoàn toàn thư mục đó khỏi hệ thống tính toán định tuyến của Next.js. Thường được trưng dụng để lưu trữ các thành phần UI dùng chung, các custom hook hoặc cấu hình thư viện cục bộ.

```

Quản lý tài nguyên dùng chung

app/
├── _components/      ← An toàn tuyệt đối, không bao giờ là một URL
│   ├── Navbar.tsx
│   └── Button.tsx
├── _lib/
│   └── prisma.ts      ← Khởi tạo kết nối Database
└── products/
    └── page.tsx        → /products

```

5. Tính Năng Nâng Cao: Định Tuyến Song Song & Đánh Chặn (Advanced Routing)

Được thiết kế nhằm giải quyết các mô hình UI phức tạp như ứng dụng bảng thông tin (Dashboard) đa luồng hoặc hiển thị cửa sổ con (Modal popup) đồng thời duy trì URL riêng biệt phục vụ chia sẻ liên kết trực tiếp.

Parallel Routes (Định tuyến song song qua ký tự @)

Hỗ trợ kết xuất nhiều trang độc lập cùng lúc trong cùng một Layout phân cấp. Các phân đoạn này được truyền trực tiếp vào Layout dưới dạng các `props` độc lập.

```

app/layout.tsx (Nhận các slot qua tham số)

export default function DashboardLayout({
  children,
  analytics, // Ảnh xạ trực tiếp từ app/@analytics/page.tsx
  team,      // Ảnh xạ trực tiếp từ app/@team/page.tsx
}: {
  children: React.ReactNode;
  analytics: React.ReactNode;
  team: React.ReactNode;
}) {
  return (
    <div className="grid-layout">
      <div className="main-feed">{children}</div>
      <div className="side-panel">
        {analytics}
        {team}
      </div>
    </div>
  );
}

```

Intercepted Routes (Định tuyến đánh chặn)

Cho phép ứng dụng tải một route mới từ một phân vùng khác hiển thị ngay tại màn hình hiện tại mà không làm mất bối cảnh ngữ cảnh cũ. Các ký hiệu đánh chặn bao gồm:

- `(. )` : Đánh chặn phân đoạn thuộc **cùng cấp bậc thư mục**.
- `(. . )` : Đánh chặn phân đoạn thuộc **trên một cấp bậc thư mục**.
- `(. . . )` : Đánh chặn phân đoạn bắt đầu tính từ thư mục **gốc gốc** `app/` .

ỨNG DỤNG THỰC TIỄN ĐIỂN HÌNH (MÔ HÌNH HIỂN THỊ MODAL HÌNH ẢNH)

Khi người dùng đang lướt xem danh sách (Feed) và click chọn một hình ảnh, giao diện popup Modal sẽ hiện lên lập tức thông qua cơ chế Intercept, URL cập nhật thành `/photo/123` . Tuy nhiên, nếu người dùng thực hiện tải lại trang (Reload) hoặc sao chép liên kết này mở ở tab mới, Next.js sẽ thông minh bỏ qua bộ đánh chặn để hiển thị toàn bộ trang chi tiết độc lập.

6. So Sánh Bản Chất Giao Diện Khung: Layouts & Templates

Việc hiểu rõ vòng đời hoạt động giúp tối ưu hóa hiệu năng và quản lý chính xác trạng thái hoạt động của các thành phần UI.

Tiêu chí	Bản chất <code>layout.tsx</code>	Bản chất <code>template.tsx</code>
Bảo toàn trạng thái	Có. Khởi tạo một lần duy nhất và duy trì nguyên trạng thái (State) khi chuyển trang con.	Không. Bị hủy hoàn toàn (Unmount) và tái khởi chạy lại từ đầu (Re-mount) khi chuyển trang.
Hiệu ứng DOM	Không kích hoạt lại các hiệu ứng lặp lại hoặc chu kỳ dữ liệu cũ.	Phù hợp để áp dụng các hiệu ứng chuyển trang (CSS Animation) hoặc đếm lượt xem (Analytics).
Trường hợp dùng	Thanh trình đơn điều hướng (Navbar), Thanh công cụ bên lề (Sidebar).	Biểu mẫu phản hồi, Theo dõi hành vi (Pageview Analytics trackers).

Hệ Thống Metadata API Toàn Diện

Next.js hỗ trợ khai báo Metadata mạnh mẽ để gia tăng điểm SEO, cho phép thiết lập cả cấu hình dạng Tĩnh hoặc linh hoạt dạng Động thông qua hàm `generateMetadata` :

```
// 1. Cấu hình Metadata dạng Động dựa vào tham số đầu vào
export async function generateMetadata({ params }: { params: { slug: string } }) {
  const product = await getProduct(params.slug);

  return {
    title: product.name,
    description: product.summary,
    openGraph: {
      images: [product.thumbnailUrl],
    },
  };
}
```